

```

import crypto from 'node:crypto';
import { q, one } from '../db.js';
import { getPackage } from './registry.js';
import { deviceFor } from '../drivers/android.js';
import { emit } from './events.js';

// One workspace per business. The Android instance and the container sit on
// the same host and are separate workloads; the container holds the private
// data and the Android instance holds the logged-in apps.
export async function provision({ businessId, region = 'us-east' }) {
  const existing = await one(`select * from workspace where business_id = $1 and
    if (existing) return existing;

  const android = await one(
    `insert into workspace (business_id, kind, region, state, persistent_volume,
      values ($1, 'cloud_android', $2, 'ready', $3, now()) returning *`,
    [businessId, region, `vol-${businessId.slice(0, 8)}`]
  );
  await one(
    `insert into workspace (business_id, kind, region, state, persistent_volume,
      values ($1, 'container', $2, 'ready', $3, now()) returning *`,
    [businessId, region, `vol-${businessId.slice(0, 8)}-svc`]
  );

  deviceFor(android.id);
  await emit({ businessId, topic: 'workspace.provisioned', payload: { workspaceId
  return android;
}

export async function androidWorkspace(businessId) {
  return one(`select * from workspace where business_id = $1 and kind = 'cloud_an
}

// Installing a channel app puts the real app on the business's own device and
// registers its map. Nothing is scraped from outside; the business is logged in.
export async function installOnDevice({ businessId, packageKey, accountLabel, ini
  const ws = await androidWorkspace(businessId);
  if (!ws) throw new Error('no android workspace for this business');

  const pkg = getPackage(packageKey);
  if (!pkg) throw new Error(`no such package: ${packageKey}`);
  const m = pkg.manifest;
  if (m.kind !== 'channel_app') throw new Error(`${packageKey} is not a channel a

```

```

await one(
  `insert into appmap (package_key, version, android_package, carries, routes)
  values ($1,$2,$3,$4::jsonb,$5::jsonb)
  on conflict (package_key, version) do update set routes = excluded.routes,
    carries = excluded.carries returning *`,
  [m.key, m.version, m.androidPackage, JSON.stringify(m.carries ?? []), JSON.st
);

const device = deviceFor(ws.id);
device.installApp(m.androidPackage, initialScreen);
device.login(m.androidPackage, accountLabel);

const row = await one(
  `insert into device_app (workspace_id, business_id, package_key, android_pack
  values ($1,$2,$3,$4,$5,true, now())
  on conflict (workspace_id, package_key) do update set logged_in = true,
    account_label = excluded.account_label, last_seen = now() returning *`,
  [ws.id, businessId, m.key, m.androidPackage, accountLabel]
);

// Every canonical key this app carries starts out unknown, not in sync.
const conn = await one(
  `insert into connection (business_id, provider_key, display_name, external_id
  values ($1,$2,$3,$4)
  on conflict (business_id, provider_key, external_id) do update set status =
  returning *`,
  [businessId, m.key, m.name, accountLabel]
);

for (const key of m.carries ?? []) {
  await one(
    `insert into channel_sync_state (business_id, connection_id, key, in_sync)
    values ($1,$2,$3,false)
    on conflict (connection_id, key) do nothing returning *`,
    [businessId, conn.id, key]
  );
}

await emit({ businessId, topic: 'device.app_installed', payload: { packageKey,
return { deviceApp: row, connection: conn };
}

export async function removeFromDevice({ businessId, packageKey }) {
  const ws = await androidWorkspace(businessId);
  await q(`delete from device_app where workspace_id = $1 and package_key = $2`,
  await q(`update connection set status = 'disconnected' where business_id = $1 a
    [businessId, packageKey]);
  return { removed: true };
}

```

```

}

// The dashboard's live view. Same device the automations drive.
export async function openSession({ ctx, mode = 'view' }) {
  const ws = await androidWorkspace(ctx.businessId);
  if (!ws) throw new Error('no workspace');
  const token = crypto.randomBytes(16).toString('hex');
  return one(
    `insert into device_session (workspace_id, business_id, opened_by, mode, stre
    values ($1,$2,$3,$4,$5,$6) returning *`,
    [ws.id, ctx.businessId, ctx.person?.id ?? null, mode,
    `/stream/${ws.id}?t=${token}`, token]
  );
}

export async function closeSession({ businessId, sessionId }) {
  return one(
    `update device_session set state = 'closed', closed_at = now()
    where id = $1 and business_id = $2 returning *`, [sessionId, businessId]
  );
}

export async function screenshot(businessId) {
  const ws = await androidWorkspace(businessId);
  const { resolve } = await import('./providers.js');
  const executor = await resolve({ slot: 'workspace.executor', businessId });
  if (!executor) throw new Error('no workspace executor bound');
  return executor.module.screenshot({ workspace: ws });
}

```